VC6

專題發表與交接

VC6 — Handover & Presentation

六週的最後一週。這一週要回答的是：
這個東西，別人接得下去嗎？

Vibe Coding 專班 · 第 6 週 · 不分軌 | 謝楠楨

這一週結束時，你手上會多出三樣東西

1 一份交接文件

- README 四段：這是什麼／怎麼跑起來／
  資料從哪來／已知的限制
- 而且要有另一個人照著跑起來過
- 期末專題五項交付物的最後一項
- 跑不起來 → 就是文件不夠

2 一份 8 分鐘的發表

- 問題是什麼／做了什麼／怎麼驗證的／
- 還有什麼不確定
- 第四段不能跳過
- 評分佔期末的 6%
- 現場會被問「為什麼相信它是對的」

3 一份課後反思

- 我在五級光譜的哪一級?
- 是什麼限制了我——工具、我的驗證能力、
  還是任務的風險?
- 這一題從第 1 週問到現在
- 六週後的答案應該不一樣了

第 1 項的驗收標準很特別：不是「文件寫得好不好」，而是「另一個人照著做，跑不跑得起來」。這是唯一客觀的標準——文件的品質，只能由讀的人來判斷。

本週地圖：哪幾頁要讀、哪幾頁要動手

頁	段落	怎麼用	預估時間
4–11	PART 01　交接：讓別人接得下去	讀。 README 四段與已知的限制	25 分
12–16	PART 02　同儕審查與發表	讀。三點式回饋與發表結構	15 分
17–22	PART 03　動手做（Step 1–5）	寫 README 、找人實測、準備發表	60–90 分
23–27	PART 04　下一步的三條路	讀。六週之後你可以往哪裡走	15 分
28–30	自我檢核、期末交付與回顧	交件前一定要看	10 分

這一週的動手部分比前幾週少，因為主體是「把東西整理成別人接得下去的樣子」。但不要低估它——Step 2 「請另一個人照著 README 跑一次」通常會抓出你完全沒想到的問題。

PART 01

交接：讓別人接得下去

Handover

「做完了」跟「交得出去」是兩件事。
這一節處理後者——而它比你想的重要得多。

為什麼「只有你會用」等於沒做完



第二列最容易被低估：交接文件最常見的讀者不是別人，是三個月後的你自己。那時候你會完全忘記「為什麼低收要寫 0」「這個數字哪裡查來的」——而 README 是唯一會記得的地方。

交接要留下的三樣東西

第③層　規則檔

CLAUDE.md ／ .cursorrules：給下一個用 AI 改它的人

- 技術限制
- 不可以做什麼

第②層　環境設定

.env.example 、需要哪些帳號與工具

- 不含任何真實金鑰

第①層　README

這是什麼／怎麼跑起來／資料從哪來／已知的限制

- 最重要的一份
- 四段

只做一樣的話，做哪一樣

README 。它是唯一「接手的人一定會打開」的檔案——其他的都是它的補充

第②層在做什麼

回答「我需要準備什麼才跑得起來」。本課程的專案幾乎不需要準備，但要寫明「不需要」

第③層是 Vibe Coding 特有的

下一個人也會用 AI 改這個專案。把你的技術限制留給他，他就不會被 AI 帶去用框架重寫

期末專題要交幾層

至少第①層。第②③層有做的話會加分，但不是必要

第③層值得多說一句：這是傳統交接文件沒有的一項。既然接手的人也會用 AI，那麼「你當初對 AI 下的限制」就是專案知識的一部分——不留下來的話，他的第一次修改就可能把整個技術棧換掉。

README 四段：一份完整的範例（照這個深度寫）

README.md

```
#  長照給付試算器

線上版本：https://你的帳號.github.io/ltc-calculator/

##  這是什麼

給照管專員與家屬用的單頁試算工具。輸入 CMS 等級
與身分別，算出每月給付上限、部分負擔比率、
民眾自負額與政府補助。手機也能正常顯示。

##  怎麼跑起來

不需要安裝任何東西。
1.  下載或 clone 這個資料夾
2.  用瀏覽器打開 index.html（直接雙擊）
3.  要跑測試的話，打開 test.html

##  資料從哪來

衛生福利部「長期照顧服務申請及給付辦法」…

##  已知的限制

（下一頁詳細說明）
```

1 第一行就放網址

讀的人最想知道的是「長什麼樣」
放最上面，一眼看到

2 「這是什麼」寫給誰看

寫給完全不認識這個專案的人
三到四句，不要寫技術細節

3 「怎麼跑起來」要能照做

編號步驟，每一步都是動作
「不需要安裝任何東西」這一句很重要

4 「資料從哪來」不能省

這是專業判斷的依據
半年後你自己也會需要它

四段的順序是設計過的：網址 → 這是什麼 → 怎麼跑 → 資料來源 → 已知的限制。讀的人的問題順序就是這樣——先想看看長什麼樣，再決定要不要深入。

「已知的限制」： README 最重要的一段

為什麼它最重要	說明
它是你唯一誠實的地方	前面三段都在說「它能做什麼」。只有這一段在說「它不能做什麼」—— 而後者才是接手的人真正需要知道的
它決定對方敢不敢用	一份沒有限制說明的文件，讀的人只有兩種反應： 全盤相信（危險），或完全不敢用（浪費）
它是專業的證明	說得出自己的東西哪裡不足，比說它有多好更有說服力。 這一點在任何專業領域都成立
期末評分看這一段	「可交接」佔期末 8%，而這一段是主要的評分依據。 寫「目前沒有已知限制」＝該項未通過
寫幾條	至少三條。任何一個工具都有三件以上做不到的事—— 寫不出三條，代表你還沒想清楚

這一段跟 VC3 的驗證報告第四段是同一件事的兩個版本：驗證報告寫給評分的人看，README 寫給接手的人看。內容可以重疊——如果你 VC3 那一份寫得好，這裡直接改寫就好。

已知的限制：一份完整範例（六條）

#	限制	為什麼要寫出來
1	只算「照顧及專業服務」這一類給付， 不含交通接送、輔具、喘息服務	使用者可能誤以為這是「全部的額度」
2	算的是「額度上限」，不是照管中心 核定的實際金額	家屬可能誤解為「我可以領到這麼多」
3	給付額度寫死在程式碼裡，年度調整時 不會自動更新，且沒有任何過期提醒	接手的人一定要知道這件事—— 否則它會在某一年開始安靜地算錯
4	沒有處理 CMS 1 級（不核發此類給付）， 但畫面上沒有說明原因	使用者選不到 1 級時會困惑
5	沒有做無障礙檢驗（鍵盤操作、色彩對比）， 而實際使用者多為高齡者與家屬	這是一個明顯的缺口，要誠實寫出來
6	未經任何專業單位審核。 介面上的「僅供參考」標示不得移除	這一條是紅線，必須寫在最後

注意第 3 條與第 6 條：它們不是「這個工具做得不夠好」，而是「使用它的人必須知道的前提」。一份好的限制清單，讀完之後對方應該能自己判斷「這個工具在什麼情況下可以信、什麼情況下不行」。

第②③層：環境設定與規則檔

檔案	寫什麼	本課程專案的實際內容
.env.example	需要哪些金鑰或設定，但不含真實的值	本專案不需要任何金鑰。 但仍建議放一個空檔並註明「本專案不需要金鑰」
README 的「需要準備什麼」	要裝什麼、要申請什麼帳號	「不需要安裝任何東西，用瀏覽器打開即可」——這一句本身就是很有價值的資訊
CLAUDE.md 或 .cursorrules	給下一個用 AI 改這個專案的人： 技術限制、不可以做什麼、專案的領域背景	「單一 HTML ＋ calculator.js，不要用框架」 「額度數字不可自行假設，須查衛福部公告」
版本說明	這一版做到哪裡、跟上一版差在哪	參考 範例\ 每個資料夾裡的「版本說明 .md」

第三列的 CLAUDE.md 值得花五分鐘寫。想像接手的人打開這個專案，跟 AI 說「幫我加一個功能」——如果沒有規則檔，AI 很可能順手把整個專案改用 React 重寫。你的限制留下來，就能擋住這件事。

交接文件的三個常見錯誤

① 寫給「已經懂的人」看

- 「執行 npm run dev 即可」——接手的人不知道要先裝 Node.js
- 判斷方法：把文件給一個完全沒看過這個專案的人，請他照做
- 他卡住的第一個地方，就是你漏寫的第一件事

② 只寫「怎麼跑」，不寫「為什麼」

- 「額度表在 calculator.js 第 10 行」——但沒寫這些數字從哪查來的
- 接手的人要更新數字時，不知道該去哪裡查、也不敢改
- → 「資料從哪來」那一段就是在解決這件事

③ 已知的限制寫「目前沒有」

- 這一句等於告訴讀的人「這份文件不可信」——任何工具都有限制
- 期末評分：這一項寫「沒有」直接視為未通過
- 寫不出來的話，回去看 VC3 的驗證報告第四段，那裡有素材

三個錯誤的共同點：你太熟悉自己的專案了，熟到看不見「哪些是別人不知道的」。這就是為什麼 Step 2 要真的找一個人照著跑一次——那是唯一能發現盲點的方法。

PART 02

同儕審查與發表

Peer Review & Presentation

你太熟悉自己的專案了，熟到看不見它哪裡令人困惑。
而使用者第一次打開時，正是那種狀態。

三點式回饋：怎麼給、怎麼收

1 一個做得好的

- 要具體，不要只說「很好」
- 「四個數字的排版讓我一眼看到自負額」
- 這一點的目的不是客套——
  它讓對方知道「哪裡不要改」
- 很多人改一改反而把優點改掉了

2 一個我看不懂的

- 具體到「哪一頁、哪個數字、哪一句話」
- 「『部分負擔比率 16%』是什麼意思？是我要付 16% 嗎？」
- 這一點最有價值——
  它是你的盲點

3 一個我會擔心的

- 「如果真的拿去用，我擔心什麼」
- 「家屬看到 36,180 會不會以為自己可以領到這麼多？」
- 這一點對應「已知的限制」——
  收到的回饋直接寫進去

收到回饋時的原則：不必馬上改，但要判斷「這是他不熟悉，還是我真的沒做好？」。而且——「他會誤解」本身就是一個發現，即使你覺得他誤解得沒道理。

找人實測：怎麼做才有效

做法	為什麼	怎麼做
不要說明， 不要在旁邊教	你一開口，測試就失效了—— 真實使用者身邊不會有你	把網址傳給他，說「你試試看」， 然後閉嘴
請他邊做邊說出來	他卡住的那一刻在想什麼， 比他最後有沒有做成功更有價值	「你現在在想什麼？」 「你覺得這個數字是什麼意思？」
找對的人	找「像你的使用者」的人， 不是找懂技術的人	照管專員、家屬、非資訊背景的同事。 自學者可以找家人
看他怎麼理解那些數字	這是醫療場域最關鍵的一點—— 數字算對了，但被理解錯了一樣有害	「所以你覺得這個人可以領多少？」 他的答案跟你的設計一樣嗎？
記下來，不要辯解	你解釋得通，不代表使用者 第一次看就懂	把每一個卡住的地方記下來， 之後判斷要不要改

> 第四列是這一頁的重點。在醫療場域，「數字算對了但被理解錯了」跟「數字算錯了」的後果可能一樣嚴重——而前者只有找人實測才發現得了，測試完全蓋不到。

8 分鐘發表：四段結構

段落	時間	說什麼	常見的錯
① **問題是什麼**	2 分	誰、在什麼情況下、遇到什麼困難。 一個具體的場景，不要講抽象的需求	花太多時間講背景， 聽的人還是不知道你做了什麼
② **做了什麼**	2 分	直接展示成品，不要只放截圖。 現場點一次給大家看	只講技術怎麼做， 不講它解決了什麼
③ **怎麼驗證的**	3 分	測了什麼、預期值從哪來、抓到什麼。 這一段時間最長，是刻意的	說「我測過了，沒問題」 ——這等於沒說
④ **還有什麼不確定**	1 分	三條以上的已知限制。 以及「什麼情況下不該用它」	跳過這一段。 現場評分會直接扣分

注意時間分配：③ 驗證佔 3 分鐘，比 ② 做了什麼還長。這對應期末評分的權重——驗證證據 14%，功能正確只有 6%。發表的結構本身就在傳達這門課的價值觀。

發表現場一定會被問到的四題（先準備好）

Q 1

你怎麼知道這些數字是對的?

答案要有出處：「我查了衛福部的給付基準」。「AI 算的」是最糟的答案

Q 2

如果現在把程式改壞一行，你會知道嗎?

「會， test.html 有 29 項測試，我實際改壞試過一次」→ VC3 Step 3-3

Q 3

這個可以正式上線嗎?

標準答案：「不行，這是原型——沒有經過資安審查、沒有稽核軌跡、沒有權限控管。但它可以用來談需求」

Q 4

你覺得它最大的問題是什麼?

這一題在測你誠不誠實。答「沒什麼問題」是最差的回答——說得出來才顯得你真的懂它

第三題與第四題是這門課六週下來最想聽到的答案。能主動說出「這是原型，不能上線」與「它最大的問題是什麼」，比做出一個很炫的東西更能證明你學會了。

PART 03

動手做： Step 1–5

Hands-on

這一週的動手不是寫程式，是把東西整理成別人接得下去的樣子。

Step 2 通常會抓出你完全沒想到的問題。

Step 1　寫 README 四段

你現在要做什麼　▶　在專案資料夾建立 README.md，照四段結構寫

可以直接抄的骨架

```
#　專案名稱

線上版本：〔你的網址〕

##　這是什麼
〔三到四句，寫給完全不認識這個專案的人〕

##　怎麼跑起來
〔編號步驟，每一步都是一個動作〕

##　資料從哪來
〔出處與版本。這一段不能省〕

##　已知的限制
〔至少三條。寫「沒有」視為未通過〕
```

你應該看到什麼

專案資料夾裡多出一個 README.md，
四段齊全。
推上 GitHub 之後，
儲存庫首頁會自動顯示它的內容。
「已知的限制」至少三條。

沒看到的話，先檢查這幾件事

① 不知道限制要寫什麼
　→ 回去看 VC3 的驗證報告第四段，
　　那裡就是素材
② 覺得寫太多會顯得做得不好
　→ 相反。說得出限制是專業的證明
③ 檔名要正確： README.md，全大寫

檔名必須是 README.md （全大寫、副檔名 .md ）， GitHub 才會自動在首頁顯示。這也是為什麼 VC0 要你把副檔名顯示出來——不然你不會發現自己存成了 README.md.txt 。

Step 2　請另一個人照著 README 實際跑一次

你現在要做什麼　▶　本週最重要的一步。不要跳過，也不要在旁邊教

怎麼做

1. 把儲存庫連結（不是網址）傳給對方
2. 說：「請照著 README 做，看能不能跑起來」
3. 然後閉嘴。他問你也先不要答，請他先自己試
4. 記下他卡住的每一個地方

自學者的替代做法

找一位家人或同事。他不必懂技術——
「跑不起來」本身就是最有價值的回饋。

或者：隔一天用另一台電腦，假裝自己是別人再跑一次。
效果比想像中好。

你應該看到什麼

對方在沒有你幫忙的情況下，
照著 README 把專案跑起來了。

卡住的地方全部記下來，
並回頭補進 README。

補完之後再請他試一次。

沒看到的話，先檢查這幾件事

① 他第一步就卡住
　→ 太好了，這就是你要的。
　　問他「你看到這一句在想什麼？」
② 找不到人
　→ 隔一天用另一台電腦自己跑一次
③ 他跑起來了但一直問你問題
　→ 每一個問題都是 README 的缺口

這一步的驗收標準是「對方跑起來了」，不是「文件寫得完整」。跑不起來，就是文件不夠——不論你覺得自己寫得多清楚。這是唯一客觀的判準。

Step 3　寫規則檔（CLAUDE.md 或 .cursorrules）

你現在要做什麼　▶　給下一個用 AI 改這個專案的人

可以直接抄（改成你的專案）

```
#  這個專案的規則

##  技術限制（請勿變更）
-  單一 index.html  ＋一個 calculator.js，不要引入任何框架
-  不要加入需要安裝的套件
-  所有資料只留在瀏覽器，不要上傳到任何伺服器

##  領域背景
-  給付額度取自衛福部長照給付基準，不可自行假設數字
-  部分負擔：一般戶 16%、中低收 5%、低收 0%

##  修改任何計算邏輯之後
-  一定要重跑 test.html，確認 29  項全綠
```

你應該看到什麼

專案資料夾裡多出一個 CLAUDE.md（或 .cursorrules）。

效果：下一個人用 AI 改這個專案時，
AI 會先讀這份規則，
不會順手把整個技術棧換掉。

沒看到的話，先檢查這幾件事

① A 軌的瀏覽器工具沒有這個功能
→ 那就寫進 README 的最後一段，
標題寫「給接手的人：修改前請先讀」
② 不知道要寫什麼
→ 把 VC1 第 21 頁你寫的技術限制
抄過來就好

這是 Vibe Coding 特有的一份文件，傳統交接不會有。理由很簡單：接手的人也會用 AI——如果不告訴 AI 你的限制，它的第一次修改就可能把整個專案改成你認不得的樣子。

Step 4　準備 8 分鐘發表

你現在要做什麼　▶　照四段結構準備，並實際計時練一次

四段與時間

① 問題是什麼　　2 分　一個具體的場景
② 做了什麼　　2 分　現場點一次，不要只放截圖
③ 怎麼驗證的　　3 分　測了什麼、預期值從哪來、抓到什麼
④ 還有什麼不確定　1 分　三條以上

一定要做的兩件事

① 實際計時練一次。八分鐘比你想的短
② 準備好回答第 16 頁那四個問題

你應該看到什麼

一份四段的簡報或講稿，
而且你實際講過一次、計過時。

第 ③ 段最長（3 分鐘），
第 ④ 段不能跳過。

成品要能現場打開來點。

沒看到的話，先檢查這幾件事

① 八分鐘講不完
　→ 砍第 ① 段。背景講太多是最常見的問題
② 不知道第 ③ 段要說什麼
　→ 直接照 VC3 的驗證報告四段講
③ 覺得第 ④ 段會讓自己看起來不好
　→ 相反。跳過它才會被扣分

「實際計時練一次」這件事常被跳過，但它是差別最大的一件事。沒練過的人幾乎一定超時，而超時通常是砍掉第 ③④ 段——恰好是評分佔比最高的兩段。

Step 5　填課後反思

你現在要做什麼　▶　回答三個問題，其中兩個從第 1 週就開始問

三個問題

① 我在五級光譜的哪一級？（VC1　第 7　頁）

② 是什麼限制了我停在那一級？
　工具能力？我的驗證能力？還是任務的風險？

③ 六週前的我，跟現在的我，判斷方式差在哪？

第 ③ 題的參考角度

- 我現在會先問「錯了會怎樣」嗎？
- 我現在敢讓 AI　做大幅修改嗎？為什麼？
- 我現在會不會先查證數字再貼給它？

你應該看到什麼

三題各寫三到五句。

沒有標準答案，
但每一題都要說得出理由。

第 ③ 題是這門課真正想問的——
六週後你的判斷方式應該不一樣了。

沒看到的話，先檢查這幾件事

① 覺得自己好像沒進步
　→ 回去看你第 1 週交的規格。
　　通常會嚇一跳
② 第 ② 題想不出來
　→ 三個選項各想一個具體的例子

第 ③ 題是整個課程的收尾。六週學到的工具與指令會過時，但「先問錯了會怎樣、先確認自己是對的、說得出哪裡不確定」這三個習慣不會——它們才是這門課真正的產出。

PART 04

下一步的三條路

What's Next

六週結束了。你現在做得出東西，也判斷得出它能不能用。
接下來可以往三個方向走。

三條路，看你的興趣與工作需要

1 把 Agent 寫出來

- 選修延伸模組（＋ 2 週）
- 讓 AI 自己決定用哪個工具、自己執行
- 本課程的 M8 （Dify）是拖拉版，
  這一條是程式碼版
- 前提：要先有測試能力（VC3）
- 因為 Agent 寫錯的代價更大

2 資料分析自動化

- 接回資料科學課程（M1–M6）
- 用 Vibe Coding 寫資料清理與報表腳本
- 重複性高、邏輯明確——最適合的一類
- 但資料規範仍然適用：
  不碰真實個案資料
- M0 的 AI Studio 也是一個選項

3 在單位裡推動

- 從一個小案子開始，不要一次提大的
- 先找「重複、明確、低風險」的痛點
- 拿原型去談需求，比拿文字規格有效
- 要先談的人：資訊室、個資窗口、
  以及那個真正會用它的同事
- VC5 第 28 頁有一頁講稿

第 3 條路最需要的不是技術，是 VC5 第 28 頁那些「怎麼回答別人」的說法。而你手上已經有了：驗證報告、成本估算、 README 、已知的限制——這四樣東西就是你在會議上的底氣。

MCP：把外部工具接給 AI 用（延伸知識）

項目	說明
它是什麼	Model Context Protocol 。一套統一的規格，讓 AI 助理能連接 外部的資料源與工具——資料庫、檔案系統、 API 、瀏覽器
為什麼你已經懂它了	這跟 M8 的 Dify Agent 「工具調用」是同一個概念，只是換了場景。 Dify 中你替 Agent 掛上搜尋工具； MCP 中你替編碼助理掛上資料庫
共同的原則	工具描述要寫清楚，否則 AI 會在錯的時機調用它。 這一點跟你這六週學的「把需求說清楚」是同一件事
這門課要用嗎	不必。列在這裡是因為你之後一定會聽到這個詞—— 知道它是什麼類別就夠了

> 這一頁的目的只是「聽到不會愣住」。真的要用 MCP 的時候，你會發現它跟你已經會的東西差別不大——最難的部分（把需求與限制說清楚）你已經練了六週。

六週之後，你實際具備什麼

判斷力

依風險決定要不要放手、驗不驗得出來

・這一項會跟著你

交付能力

公開網址、儲存庫、 README 、可交接

・別人接得下去

驗證能力

讀懂產出、訂驗收條件、寫測試、說出不確定

・這門課的中軸

生成能力

把需求寫成規格，讓 AI 做出可運行的東西

・最容易取得的一項

為什麼生成能力放最下面

它最容易取得——任何一個人花一小時都學得會。而且它最快過時，因為工具每半年就換一輪

判斷力為什麼在最上面

它不依賴任何特定工具，也不會過時。「錯了會怎樣、我驗不驗得出來」這兩個問題十年後一樣適用

哪一項最難補

驗證能力。它需要領域知識 ＋ 方法 ＋ 習慣三者兼具——而前者你本來就有

六週最大的改變

從「AI 說可以就可以」，變成「我有辦法判斷它可不可以」

這一頁是六週的總結。如果只能記住一句話：這門課教的不是「怎麼叫 AI 寫程式」，而是「怎麼在不會寫程式的前提下，判斷它寫的東西能不能用」。

帶得走的六個習慣（跟工具無關）

習慣	來自	為什麼它不會過時
先問「錯了會怎樣」	VC1 五級光譜	這是任何風險決策的第一題，不限於 AI
動手前先寫清楚要什麼	VC1 五行規格	需求說不清楚，換誰做都會做錯
給證據不給形容詞	VC2 除錯	這是跟任何人溝通問題的通則
先確認「我是對的」	VC2 Step 2-2	AI 不會質疑你。把關的只有你自己
改完確認「別的地方沒壞」	VC3 測試	任何修改都有副作用，這是通則
說得出自己不知道什麼	VC3 驗證報告	這是專業的定義，不是弱點

> 六個習慣沒有一個提到任何工具的名字。這是刻意的——三年後 Lovable、Cursor、Claude Code 可能都換了一輪，但這六個習慣一個都不會變。

本週自我檢核

Q 1

README 四段是哪四段？哪一段最重要、為什麼？

這是什麼／怎麼跑起來／資料從哪來／已知的限制。最後一段是你唯一誠實的地方 → 第 7 頁與第 8 頁

Q 2

「交接文件寫得好」的客觀判準是什麼？

另一個人照著做，跑不跑得起來。不是「你覺得寫得清楚」→ 第 19 頁

Q 3

為什麼 Vibe Coding 的交接要多一份規則檔？

因為接手的人也會用 AI。不留下限制，它可能把整個技術棧換掉 → 第 10 頁與第 20 頁

Q 4

六週前的你，跟現在的你，判斷方式差在哪？

這一題沒有標準答案，但一定要說得出來 → 第 22 頁的課後反思

第四題是這門課真正的期末考。前三題答得出來代表你記住了內容；第四題答得出來，代表這六週真的改變了你做事的方式。

期末專題：五項交付物怎麼對應到評分（合計 40%）

評分項目	對應交付物	拿到滿分的條件	佔比
① **規格書**	① 規格書（VC1）	五項齊全，驗收條件可測、可執行、與成品一致	6%
② **功能正確**	② 公開網址（VC4）	別人（含手機）打得開、跑得動； 21 種組合與邊界值都正確	6%
③ **驗證證據**	④ 驗證報告（VC3）	有測試、有審查紀錄、有覆蓋網格，且明確寫出還有哪些不確定	14%
④ **可交接**	③ 儲存庫 + ⑤ 交接文件（VC4 / VC6）	另一人照 README 能獨立跑起來；已知限制寫得誠實；無金鑰外流（發現金鑰、密碼或真實個案資料外流 → 本項直接零分）	8%
⑤ **發表**	8 分鐘發表（VC6）	四段完整，且答得出「為什麼相信它是對的」	6%

全課程 100% ＝每週交付 40%（W1–W5 各 8%）＋期末專題 40%（上表）＋課堂參與與同儕審查 20%。注意 ③ 驗證證據 14%，是 ② 功能正確 6% 的兩倍以上——一個功能只做了一半但把限制寫得清清楚楚的專案，在本課程的評分下應該高於一個看起來很完整卻說不出怎麼驗證的專案。

六週回顧

1. 難處不在「叫 AI 寫程式」，而在「你到底要什麼」（VC1）
2. AI 看不到你的螢幕，你是它唯一的感官（VC2）
3. 測試只保證「跟你想的一樣」，不保證「你想的是對的」（VC3）
4. 改壞了要回得去，做好了要別人打得開（VC4）
5. 「存起來」不是一個功能，是一個治理決定（VC5）
6. 說得出自己不知道什麼，才叫真的做完（VC6）

下一步　想繼續的話：選修延伸模組「用 Vibe Coding 做 Agent」（＋2 週）。但先做完第 22 頁的課後反思——六週學到的工具會過時，那三個習慣不會。